

Mountain Car

1. Introducción y contexto

En este ejercicio implementamos dos enfoques de Aprendizaje por Refuerzo: Q-Learning tabular y Deep Q-Network (DQN). La comparación permite observar cómo cambia el proceso de aprendizaje cuando la función de valor-acción $Q(s, a)$ se representa mediante una tabla discreta (Q-Learning tabular) o se aproxima utilizando una red neuronal (DQN).

Sobre el entorno MountainCar-v0 de Gymnasium, se completaron los componentes principales de ambos agentes, se ejecutaron sus procesos de entrenamiento y se registraron los resultados obtenidos. En el caso de Q-Learning, fue necesario discretizar el espacio de estados (era continuo) para construir y actualizar la tabla Q. Para DQN, se utilizó una red neuronal que trabaja directamente con las variables continuas del entorno, junto con mecanismos como experience replay y una target network.

Finalmente, los dos enfoques se comparan a partir de las curvas y métricas obtenidas durante el entrenamiento y la evaluación, considerando el desempeño alcanzado, la estabilidad, los episodios requeridos para aprender y las principales ventajas y limitaciones observadas.

2. El problema

El agente debe controlar un carrito ubicado en un valle entre dos colinas. Debido a que su motor es débil para alcanzar directamente la cima derecha, la única forma de alcanzar la bandera es mecerse hacia atrás y adelante para acumular impulso. El agente debe aprender una secuencia de acciones que le permita acumular impulso hasta alcanzar la posición objetivo.

2.1 Estado

En cada instante t el estado del agente está definido por:

$$s_t = (p_t, v_t)$$

Donde MountainCar-v0 tiene un espacio de estados continuo de dos dimensiones

- p_t : posición del vehículo, con $p_t \in [-1.2, 0.6]$
- v_t : velocidad del vehículo, con $v_t \in [-0.07, 0.07]$

2.2 Acciones

En cada paso, el agente selecciona una acción a_t perteneciente al conjunto:

$$A = \{0, 1, 2\}$$
$$a_t = \begin{cases} 0 & \text{acelerar hacia la izquierda} \\ 1 & \text{no acelerar} \\ 2 & \text{acelerar hacia la derecha} \end{cases}$$

2.3 Objetivo

El objetivo es alcanzar la cima derecha del entorno utilizando la menor cantidad posible de pasos.

$$p_t \geq 0.5$$

2.4 Recompensa

La recompensa es -1 en cada paso hasta que el episodio termina, con un tope de 200 pasos. El retorno acumulado de un episodio puede expresarse como:

$$G = \sum_{t=0}^{T-1} r_t$$

donde T corresponde al número de pasos ejecutados hasta alcanzar la meta o finalizar el episodio.

Como cada paso tiene una penalización de -1, **maximizar el retorno equivale a minimizar la cantidad de pasos requeridos para alcanzar la bandera.**

Por ejemplo, un episodio que finaliza en 120 pasos obtiene un retorno de -120, mientras que un agente que no alcanza la meta dentro del límite de 200 pasos obtiene: $G = -200$

3. Q-Learning tabular

Se implementó un agente de Q-Learning tabular. Dado que el entorno utiliza variables continuas, el primer paso consistió en discretizarlas mediante intervalos o bins. Para ello, cada dimensión del estado se dividió en 20 intervalos, generando una rejilla de $20 \times 20 = 400$ estados discretos sobre los cuales se construyó la tabla Q.

```
def discretize(self, obs: np.ndarray) -> tuple:
    return tuple(
        int(np.digitize(value, edges))
        for value, edges in zip(obs, self._bins)
    )
```

- **discretize** recibe la observación continua del entorno —posición y velocidad— e identifica a qué intervalo pertenece cada valor. Como resultado, transforma el estado continuo en una tupla de dos índices discretos, por ejemplo (7, 12), que se utiliza como referencia dentro de la tabla Q.

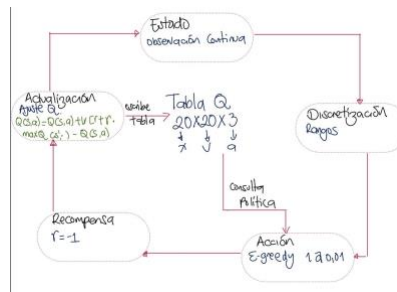
```
def select_action(self, state: tuple, *, deterministic: bool = False) -> int:
    if not deterministic and random.random() < self.epsilon:
        return random.randrange(self.n_actions) # explorar
    return int(np.argmax(self.q_table[state])) # explotar
```

- **select_action** balancea explorar (probar algo al azar) y explotar (usar lo mejor ya conocido)
- **epsilon** es la probabilidad de explorar en cada paso: empieza en 1.0 (el agente no sabe nada) y decae gradualmente hasta un piso de 0.01.
- El **parámetro deterministic** fuerza siempre la rama de explotación sin importar qué tan alto esté epsilon; es el modo que usan la evaluación (load --eval) y el renderizado, donde no se quiere ruido aleatorio.

```
def _update(self, state, action, reward, next_state, terminated):
    if terminated:
        target = reward
    else:
        target = reward + self.gamma * np.max(self.q_table[next_state])

    td_error = target - self.q_table[state][action]
    self.q_table[state][action] += self.lr * td_error
```

- **_update** ajusta la tabla Q después de cada paso, acercando el valor estimado hacia el target calculado con la ecuación de Bellman. El target combina la recompensa del paso con el mejor valor futuro posible desde el siguiente estado, descontado por gamma; la diferencia con el valor actual (error TD) se multiplica por la tasa de aprendizaje (lr). Cuando el episodio termina (terminated=True) no existe estado siguiente, así que el target es simplemente la recompensa. La actualización se escribe directamente en self.q_table[state][action] y no en una copia local, para que el aprendizaje quede registrado en la tabla real del agente.



Verificación y resultados

Una primera corrida de 1.000 episodios permitió verificar que el agente estaba explorando progresivamente el espacio de estados, aunque no lograba alcanzar la meta. Posteriormente, el entrenamiento se extendió hasta 21.000 episodios. Al finalizar, el agente había visitado 297 de los 400 estados discretizados. Para evaluar la política aprendida se ejecutaron 100 episodios, en los cuales el agente alcanzó la bandera en todos los casos.

```
PS C:\Users\marlo\OneDrive\Escritorio\MA\Simulación\Trabajo_2\mountain_car> uv run mountaincar_train qlearning --episode
c 20000
Episode 100/20000 | Avg Reward: -200.00 | Epsilon: 0.8769 | States visited: 179
Episode 200/20000 | Avg Reward: -200.00 | Epsilon: 0.8487 | States visited: 218
Episode 300/20000 | Avg Reward: -200.00 | Epsilon: 0.8220 | States visited: 215
Episode 400/20000 | Avg Reward: -200.00 | Epsilon: 0.4963 | States visited: 218
Episode 500/20000 | Avg Reward: -200.00 | Epsilon: 0.4723 | States visited: 237
Episode 600/20000 | Avg Reward: -200.00 | Epsilon: 0.4492 | States visited: 247
Episode 700/20000 | Avg Reward: -199.04 | Epsilon: 0.4273 | States visited: 255
Episode 800/20000 | Avg Reward: -199.38 | Epsilon: 0.4065 | States visited: 257
Episode 900/20000 | Avg Reward: -200.00 | Epsilon: 0.3866 | States visited: 257
Episode 1000/20000 | Avg Reward: -199.01 | Epsilon: 0.3678 | States visited: 257
```

El reward medio obtenido fue de -141,1, con una desviación estándar de 6,68. El mejor episodio alcanzó un reward de -114, mientras que el peor obtuvo -150. Estos resultados muestran que el agente aprendió una política capaz

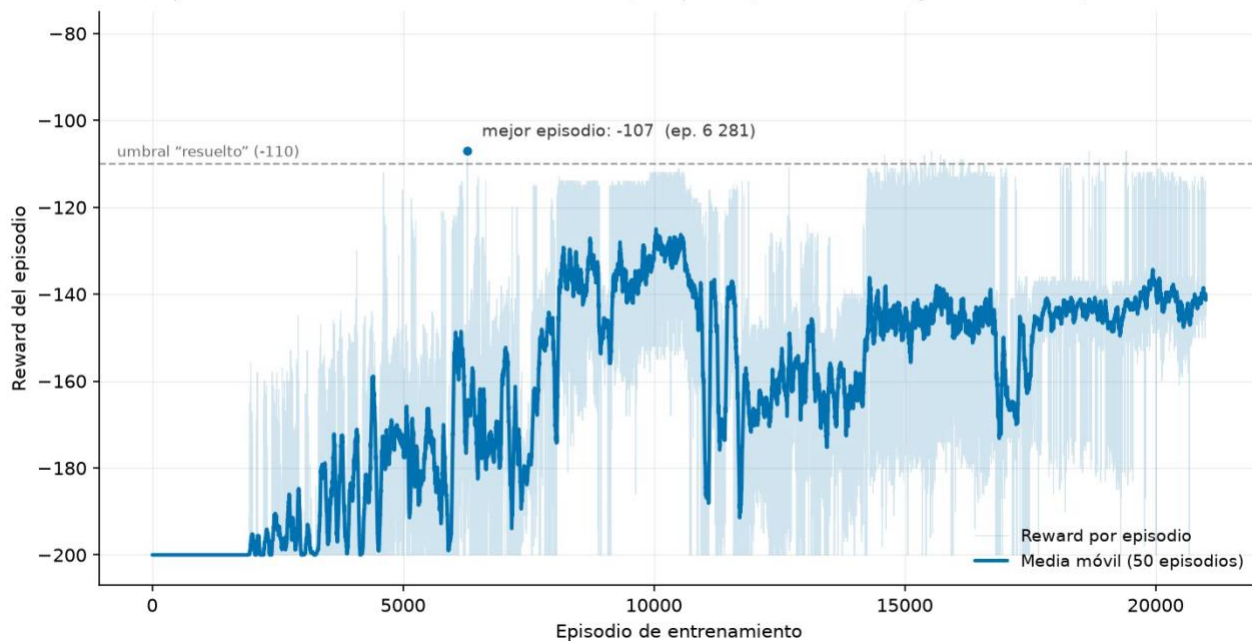
Jorge Anaya Joo | Andrés Díaz Lázaro | Nicolás Gamboa Hidalgo | Diego Ríos Gómez | Marlon Umbarila Ortega | Nataly Valbuena López | Lorena Valero González

de resolver el entorno de manera consistente, aunque con diferencias en el número de pasos requeridos entre episodios

```
{
  "episodes": 100,
  "mean_reward": -141.1,
  "std_reward": 6.67757440991862,
  "best_reward": -114.0,
  "worst_reward": -150.0,
  "reached_flag": 100,
  "training_episodes": 21000,
  "states_visited": 297
}
```

Ejercicio 1 · Q-Learning tabular

21 000 episodios de entrenamiento · evaluación determinista (100 episodios): -141.1 ± 6.7 · llega a la bandera 100/100



La gráfica muestra que el agente de **Q-Learning** empezó con recompensas cercanas a **-200** y fue mejorando a medida que avanzó el entrenamiento. El aprendizaje no fue completamente estable, porque durante varios momentos la recompensa volvió a caer, pero la media móvil sí muestra una mejora frente a los primeros episodios. El mejor resultado individual fue de **-107 en el episodio 6.281**. Después de los 21.000 episodios de entrenamiento, al evaluar el agente durante 100 episodios alcanzó la bandera en **100 de 100 casos**, con una recompensa promedio de **-141,1 ± 6,7**. Esto muestra que el agente sí logró aprender a resolver el problema de forma consistente, aunque con espacio para mejorar el número de pasos necesarios.

4. Deep Q-Network

En DQN dejamos de utilizar una tabla Q con estados discretizados y la reemplazamos por una red neuronal que estima qué tan conveniente es cada acción para un estado determinado. Esto permite trabajar directamente con la posición y la velocidad del carro, sin tener que convertirlas previamente en casillas. La red recibe como entrada los dos valores del estado: posición y velocidad.

Entrega tres valores de salida, uno por cada acción posible: acelerar a la izquierda, no acelerar o acelerar a la derecha. La acción con el valor Q más alto representa, en ese momento, la alternativa que la red considera más conveniente.

```
def __init__(self, state_dim: int, action_dim: int, hidden: int = 128) -> None:
    super().__init__()
    self.net = nn.Sequential(
        nn.Linear(state_dim, hidden),
        nn.ReLU(),
        nn.Linear(hidden, hidden),
        nn.ReLU(),
        nn.Linear(hidden, action_dim),
    )

def forward(self, x: torch.Tensor) -> torch.Tensor:
    return self.net(x)
```

La arquitectura tiene dos capas de 128 neuronas. La red recibe el estado actual del carro y va ajustando sus pesos para aprender qué acción suele generar un mejor resultado. Para entrenar, el agente guarda las experiencias que va teniendo en un replay buffer. Cada experiencia contiene el estado, la acción tomada, la recompensa recibida y el siguiente estado. Luego, toma pequeños grupos de experiencias anteriores y los utiliza para actualizar la red.

```
current_q = self.q_net(states_t).gather(1, actions_t)

with torch.no_grad():
    next_q = self.target_net(next_states_t).max(dim=1, keepdim=True).values
    target_q = rewards_t + self.gamma * next_q * (1.0 - terminateds_t)

loss = self.loss_fn(current_q, target_q)
self.optimizer.zero_grad()
loss.backward()
self.optimizer.step()

return loss.item()
```

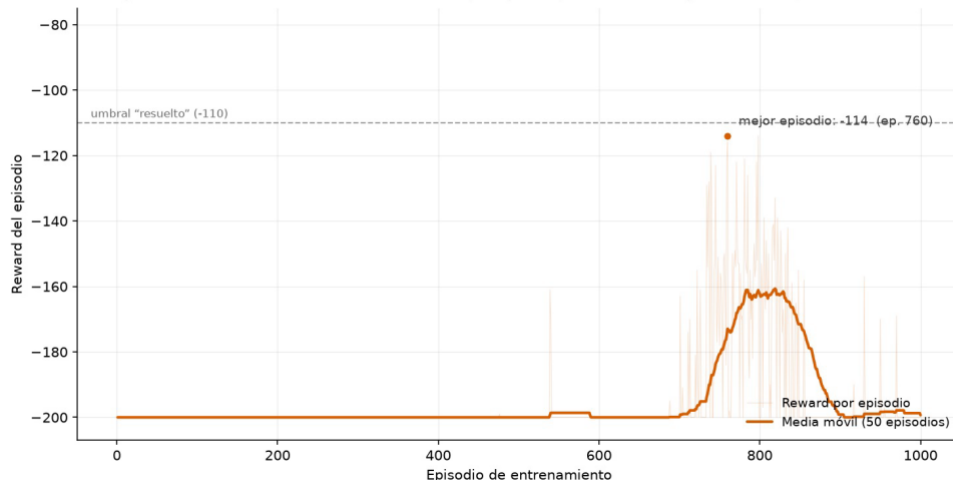
En esta parte se compara: lo que la red actualmente cree que vale la acción tomada y lo que debería valer según la recompensa obtenida y lo que podría ocurrir después. La diferencia se utiliza como error para ajustar los pesos de la red.

```
Episode 610/1000 | Avg Reward: -200.00 | Epsilon: 0.0470 | Buffer: 100000
Episode 620/1000 | Avg Reward: -200.00 | Epsilon: 0.0447 | Buffer: 100000
Episode 630/1000 | Avg Reward: -200.00 | Epsilon: 0.0425 | Buffer: 100000
Episode 640/1000 | Avg Reward: -200.00 | Epsilon: 0.0404 | Buffer: 100000
Episode 650/1000 | Avg Reward: -200.00 | Epsilon: 0.0385 | Buffer: 100000
Episode 660/1000 | Avg Reward: -200.00 | Epsilon: 0.0366 | Buffer: 100000
Episode 670/1000 | Avg Reward: -200.00 | Epsilon: 0.0348 | Buffer: 100000
Episode 680/1000 | Avg Reward: -200.00 | Epsilon: 0.0331 | Buffer: 100000
Episode 690/1000 | Avg Reward: -200.00 | Epsilon: 0.0315 | Buffer: 100000
Episode 700/1000 | Avg Reward: -200.00 | Epsilon: 0.0299 | Buffer: 100000
Episode 710/1000 | Avg Reward: -200.00 | Epsilon: 0.0285 | Buffer: 100000
Episode 720/1000 | Avg Reward: -200.00 | Epsilon: 0.0271 | Buffer: 100000
Episode 730/1000 | Avg Reward: -200.00 | Epsilon: 0.0258 | Buffer: 100000
Episode 740/1000 | Avg Reward: -200.00 | Epsilon: 0.0245 | Buffer: 100000
Episode 750/1000 | Avg Reward: -200.00 | Epsilon: 0.0233 | Buffer: 100000
Episode 760/1000 | Avg Reward: -200.00 | Epsilon: 0.0222 | Buffer: 100000
Episode 770/1000 | Avg Reward: -200.00 | Epsilon: 0.0211 | Buffer: 100000
Episode 780/1000 | Avg Reward: -200.00 | Epsilon: 0.0200 | Buffer: 100000
Episode 790/1000 | Avg Reward: -200.00 | Epsilon: 0.0191 | Buffer: 100000
Episode 800/1000 | Avg Reward: -200.00 | Epsilon: 0.0181 | Buffer: 100000
Episode 810/1000 | Avg Reward: -200.00 | Epsilon: 0.0172 | Buffer: 100000
Episode 820/1000 | Avg Reward: -200.00 | Epsilon: 0.0164 | Buffer: 100000
Episode 830/1000 | Avg Reward: -200.00 | Epsilon: 0.0156 | Buffer: 100000
Episode 840/1000 | Avg Reward: -200.00 | Epsilon: 0.0148 | Buffer: 100000
Episode 850/1000 | Avg Reward: -200.00 | Epsilon: 0.0141 | Buffer: 100000
Episode 860/1000 | Avg Reward: -200.00 | Epsilon: 0.0134 | Buffer: 100000
Episode 870/1000 | Avg Reward: -200.00 | Epsilon: 0.0128 | Buffer: 100000
Episode 880/1000 | Avg Reward: -200.00 | Epsilon: 0.0121 | Buffer: 100000
Episode 890/1000 | Avg Reward: -200.00 | Epsilon: 0.0115 | Buffer: 100000
Episode 900/1000 | Avg Reward: -200.00 | Epsilon: 0.0110 | Buffer: 100000
Episode 910/1000 | Avg Reward: -200.00 | Epsilon: 0.0104 | Buffer: 100000
Episode 920/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 930/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 940/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 950/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 960/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 970/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 980/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 990/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 1000/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Saved DQN agent to saves\dqn_ejercicio2.pt
```

```
{
  "episodes": 100,
  "mean_reward": -200.0,
  "std_reward": 0.0,
  "best_reward": -200.0,
  "worst_reward": -200.0,
  "reached_flag": 0,
  "training_episodes": 1000,
  "stickiness": 0.0,
  "target_update_freq": 10,
  "epsilon_decay": 0.995
}
```

Ejercicio 2 · DQN con exploración uniforme

1 000 episodios de entrenamiento · evaluación determinista (100 episodios): -200.0 ± 0.0 · llega a la bandera 0/100



Después de 1.000 episodios de entrenamiento, el agente permaneció la mayor parte del tiempo con recompensas cercanas a -200 . Sin embargo, hacia los últimos episodios se observa una mejora temporal, llegando a un mejor resultado individual de -114 en el episodio 760. A pesar de esto, la mejora no fue estable y la recompensa volvió a deteriorarse al final del entrenamiento. La evaluación sobre 100 episodios confirmó que el agente no había aprendido una política consistente: obtuvo una recompensa promedio de -200 , sin variación, y no alcanzó la bandera en ninguno de los 100 intentos. Esto muestra que el DQN logró encontrar de manera puntual una trayectoria favorable durante el entrenamiento, pero todavía no consiguió generalizar ese comportamiento para resolver MountainCar de forma consistente.

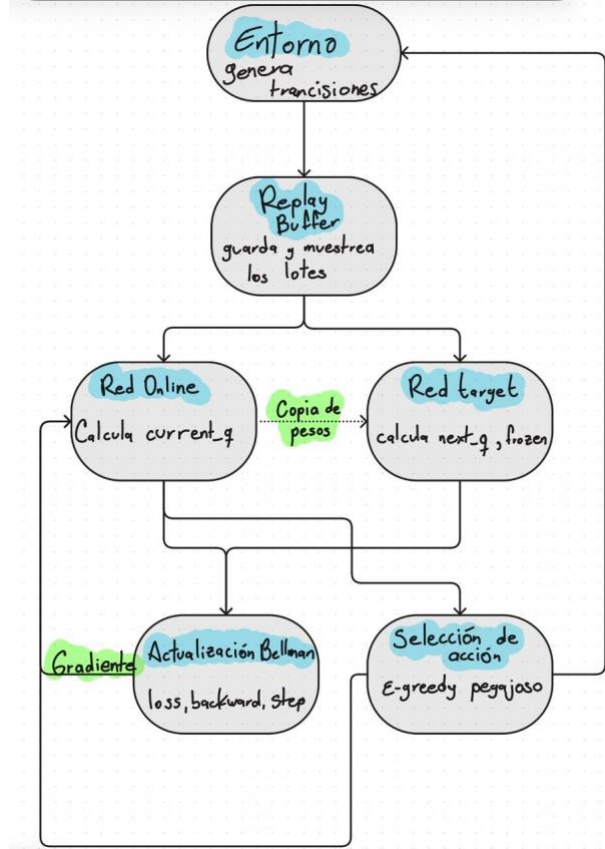
5. Deep Q Network - Stickiness

Aunque la red estaba funcionando, el resultado se mantenía en -200 , incluso después de aumentar el número de episodios. En MountainCar, cada paso entrega una recompensa de -1 , independientemente de si el carro se está acercando o alejando de la meta. Si el agente nunca alcanza la bandera, prácticamente todas las experiencias que almacena se parecen: episodios que terminan en -200 . Por esta razón, la red recibe muy poca información que le permita identificar la secuencia que puede llevarla a un mejor resultado.

El problema aparece con la exploración epsilon-greedy tradicional, porque cuando el agente decide explorar selecciona una acción aleatoria en cada paso. En MountainCar esto genera cambios constantes de dirección, cuando en realidad el carro necesita mantener el impulso durante varios pasos para ganar velocidad y poder subir. Para corregir este comportamiento se incorporó el hiperparámetro stickiness. El cual le da una probabilidad de mantener la última acción exploratoria durante varios pasos.

```
def select_action(self, state: np.ndarray, *, deterministic: bool = False) -> int:
    if not deterministic and random.random() < self.epsilon:
        if self._last_explore_action is not None and random.random() < self.stickiness:
            return self._last_explore_action
        action = random.randrange(self.action_dim)
        self._last_explore_action = action
        return action

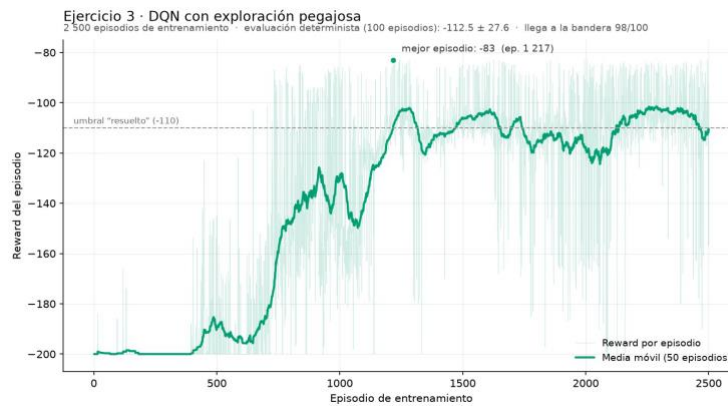
    with torch.no_grad():
        t = torch.as_tensor(state, dtype=torch.float32, device=self.device).unsqueeze(0)
        return int(self.q_net(t).argmax(dim=1).item())
```



```

Episode 2110/2500 | Avg Reward: -116.40 | Epsilon: 0.0100 | Buffer: 100000
Episode 2120/2500 | Avg Reward: -134.60 | Epsilon: 0.0100 | Buffer: 100000
Episode 2130/2500 | Avg Reward: -109.80 | Epsilon: 0.0100 | Buffer: 100000
Episode 2140/2500 | Avg Reward: -111.60 | Epsilon: 0.0100 | Buffer: 100000
Episode 2150/2500 | Avg Reward: -138.30 | Epsilon: 0.0100 | Buffer: 100000
Episode 2160/2500 | Avg Reward: -125.10 | Epsilon: 0.0100 | Buffer: 100000
Episode 2170/2500 | Avg Reward: -130.60 | Epsilon: 0.0100 | Buffer: 100000
Episode 2180/2500 | Avg Reward: -124.90 | Epsilon: 0.0100 | Buffer: 100000
Episode 2190/2500 | Avg Reward: -132.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 2200/2500 | Avg Reward: -136.70 | Epsilon: 0.0100 | Buffer: 100000
Episode 2210/2500 | Avg Reward: -147.20 | Epsilon: 0.0100 | Buffer: 100000
Episode 2220/2500 | Avg Reward: -134.10 | Epsilon: 0.0100 | Buffer: 100000
Episode 2230/2500 | Avg Reward: -129.60 | Epsilon: 0.0100 | Buffer: 100000
Episode 2240/2500 | Avg Reward: -116.80 | Epsilon: 0.0100 | Buffer: 100000
Episode 2250/2500 | Avg Reward: -104.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 2260/2500 | Avg Reward: -138.70 | Epsilon: 0.0100 | Buffer: 100000
Episode 2270/2500 | Avg Reward: -131.60 | Epsilon: 0.0100 | Buffer: 100000
Episode 2280/2500 | Avg Reward: -123.10 | Epsilon: 0.0100 | Buffer: 100000
Episode 2290/2500 | Avg Reward: -123.10 | Epsilon: 0.0100 | Buffer: 100000
Episode 2300/2500 | Avg Reward: -132.70 | Epsilon: 0.0100 | Buffer: 100000
Episode 2310/2500 | Avg Reward: -120.70 | Epsilon: 0.0100 | Buffer: 100000
Episode 2320/2500 | Avg Reward: -133.90 | Epsilon: 0.0100 | Buffer: 100000
Episode 2330/2500 | Avg Reward: -149.60 | Epsilon: 0.0100 | Buffer: 100000
Episode 2340/2500 | Avg Reward: -149.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 2350/2500 | Avg Reward: -127.20 | Epsilon: 0.0100 | Buffer: 100000
Episode 2360/2500 | Avg Reward: -146.30 | Epsilon: 0.0100 | Buffer: 100000
Episode 2370/2500 | Avg Reward: -133.10 | Epsilon: 0.0100 | Buffer: 100000
Episode 2380/2500 | Avg Reward: -104.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 2390/2500 | Avg Reward: -123.60 | Epsilon: 0.0100 | Buffer: 100000
Episode 2400/2500 | Avg Reward: -113.80 | Epsilon: 0.0100 | Buffer: 100000
Episode 2410/2500 | Avg Reward: -118.40 | Epsilon: 0.0100 | Buffer: 100000
Episode 2420/2500 | Avg Reward: -125.80 | Epsilon: 0.0100 | Buffer: 100000
Episode 2430/2500 | Avg Reward: -118.80 | Epsilon: 0.0100 | Buffer: 100000
Episode 2440/2500 | Avg Reward: -129.60 | Epsilon: 0.0100 | Buffer: 100000
Episode 2450/2500 | Avg Reward: -133.30 | Epsilon: 0.0100 | Buffer: 100000
Episode 2460/2500 | Avg Reward: -134.40 | Epsilon: 0.0100 | Buffer: 100000
Episode 2470/2500 | Avg Reward: -140.80 | Epsilon: 0.0100 | Buffer: 100000
Episode 2480/2500 | Avg Reward: -137.20 | Epsilon: 0.0100 | Buffer: 100000
Episode 2490/2500 | Avg Reward: -141.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 2500/2500 | Avg Reward: -149.10 | Epsilon: 0.0100 | Buffer: 100000
Saved DQN agent to saves\dqn_ejercicio3.pt
  
```

```
{
  "episodes": 100,
  "mean_reward": -134.21,
  "std_reward": 22.112121110377444,
  "best_reward": -89.0,
  "worst_reward": -168.0,
  "reached_flag": 100,
  "training_episodes": 4000,
  "stickiness": 0.9,
  "target_update_freq": 5,
  "epsilon_decay": 0.997
}
```



Con la exploración sticky epsilon-greedy, el DQN mejoró de forma evidente. En el entrenamiento de 2.500 episodios alcanzó un mejor reward de -83 , y en la evaluación obtuvo $-112,5 \pm 27,6$, llegando a la bandera en 98 de 100 episodios. Al extender el entrenamiento a 4.000 episodios, alcanzó la meta en 100 de 100 evaluaciones, aunque el reward promedio bajó a $-134,21 \pm 22,11$. Esto muestra que el cambio en la exploración fue clave para que el agente aprendiera, aunque más episodios no necesariamente significaron un mejor desempeño promedio.

6. Comparativa de los modelos

La gráfica permite ver diferencias claras entre los tres entrenamientos. **Q-Learning** logró aprender de forma progresiva, pero necesitó muchos más episodios y se mantuvo por debajo del umbral de -110 en su media móvil. El **DQN con exploración uniforme** presentó una mejora puntual durante el entrenamiento, pero no logró mantenerla ni resolver el entorno durante la evaluación. En cambio, el **DQN con exploración pegajosa** aprendió mucho más rápido y alcanzó valores cercanos e incluso superiores al umbral de referencia durante algunos periodos del entrenamiento.

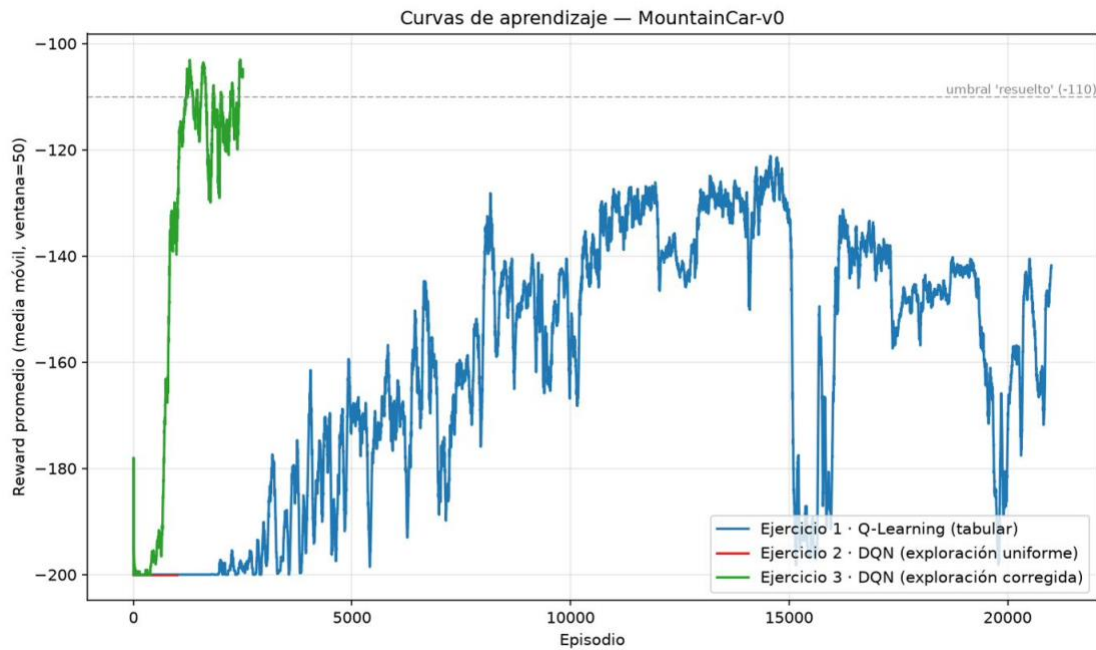


Figura 1. Reward promedio (media móvil, ventana=50) por episodio para los tres agentes entrenados en MountainCar-v0.

Criterio	Q-Learning	DQN uniforme	DQN pegajosa
Episodios de entrenamiento	21.000	1.000	2.500 / 4.000
Mejor reward observado	-107	-114	-83
Evaluación	$-141,1 \pm 6,7$	-200	$-112,5 \pm 27,6^*$
Llegó a la bandera	100/100	0/100	98/100*
Velocidad de aprendizaje	Lenta	No consolidó aprendizaje	Más rápida
Estabilidad	Alta en evaluación	Baja	Mayor variabilidad
Complejidad	Menor	Mayor	Mayor

*Resultados de la evaluación realizada después de 2.500 episodios. Al extender el entrenamiento a 4.000 episodios alcanzó **100/100**, aunque el reward medio bajó a $-134,21 \pm 22,11$.

- **Q-Learning logró resolver el entorno de forma consistente**, pero necesitó muchos más episodios de entrenamiento. Su principal ventaja fue la estabilidad obtenida en la evaluación.
- **DQN no mejoró únicamente por utilizar una red neuronal**. Con la exploración uniforme el agente no logró aprender una política estable, mientras que al modificar la estrategia de exploración el desempeño mejoró.
- El **DQN con exploración pegajosa fue el que aprendió más rápido y obtuvo el mejor reward durante el entrenamiento**, aunque presentó mayor variabilidad.